

# 架构候选方案对比表

针对校园互助平台这种业务模块多（跑腿、交易、社交）、用户基数波动大、且需要在 10 周内交付的特点，我们选择了两种最具代表性的架构方案进行对比：

- 方案 A：分层架构（Layered Architecture / Monolithic） - 经典的“大一统”模式。
- 方案 B：微服务架构（Microservices Architecture） - 现代的“解耦”模式。

## 方案 A：分层架构 (Layered Architecture)

分层架构（最常见的是 MVC 三层架构）是软件工程的经典模式。它将系统横向切分为几层，每一层只负责特定的功能，且通常只能向下调用。

表现层 (Presentation Layer): 负责展示界面、接收学生填写的跑腿订单，并将结果反馈给用户。

业务逻辑层 (Business Layer): 这是核心，负责计算积分、验证学号是否合规、判断订单状态。

数据访问层 (Data Access Layer): 负责从数据库中读取或写入用户信息、订单详情等。

这是最传统的架构模式。对于校园项目，所有功能（跑腿、失物、积分等）都运行在同一个进程中。

- 优势：开发效率极高，调试方便，不需要处理复杂的分布式事务和网络延迟。
- 劣势：代码耦合度高。如果“跑腿服务”的代码写出了 Bug 导致崩溃，整个平台（包括失物招领和积分）都会瘫痪。

## 方案 B：微服务架构 (Microservices Architecture)

微服务不再是把所有功能塞进一个大系统，而是将每个功能拆分成一个独立的、能自行运行的“微型应用”。

用户服务： 专门负责学号认证，像一个独立的校务办公室。

跑腿服务： 专门处理快递代取，像一个独立的快递点。

积分服务： 专门管钱和分值，像校园卡充值中心。

社交服务： 专门处理聊天和讨论。

将不同功能拆分为独立的服务，例如：用户服务、任务服务（跑腿）、交易服务（二手）、消息服务（社交）。服务之间通过 REST API 或消息队列通讯。

- **优势：** 模块化极强。后期如果“二手交易”流量激增，可以单独给它增加服务器资源，而不影响其他功能。
- **劣势：** 基础设施搭建成本极高。你需要处理服务注册、发现、负载均衡以及分布式数据一致性（比如积分扣除成功但任务状态没更新怎么办）。

架构方案对比表

| 对比维度     | 方案 A：分层架构 (Monolith) | 方案 B：微服务架构 (Microservices)      |
|----------|----------------------|---------------------------------|
| 开发复杂度    | 低：单一工程，逻辑直观。         | 高：涉及服务拆分、接口定义和分布式通信。            |
| 可维护性     | 中：初期容易，后期代码量大后会变得臃肿。 | 高：服务解耦，单个模块的修改不影响全局。            |
| 可扩展性     | 低：只能对整个应用进行水平扩展。     | 高：可针对特定高频模块（如跑腿）精准扩容。           |
| 性能       | 高：本地方法调用，无网络开销。      | 中：存在跨服务的网络延迟。                   |
| 团队学习成本   | 低：掌握基础 Web 开发即可上手。   | 高：需学习 Docker, API 网关, RPC 等技术栈。 |
| 10 周内可行性 | 极高：非常适合快速迭代并上线。      | 低：光是搭建底层设施可能就要花掉 3-4 周。         |

深度分析与建议

针对我们的校园互助平台，经过激烈讨论，我们最终选择 **方案 A（分层架构）**。

理由如下：

1. **交付压力：** 10 周的时间非常紧迫。微服务带来的“分布式事务”和“服务间治理”会占据大量开发时间，导致核心业务逻辑（如积分结算、评价机制）无法按时完成。
2. **业务初期规模：** 校园平台的初期并发量通常不会达到必须使用微服务的量级。
3. **折中方案：** 你们可以采用 **“模块化单体”** 的思路。在方案 A 的基础上，代码层面保持严格的模块化（例如 `com.platform.task`, `com.platform.user`），这样未来如果用户量真的突破万级，再剥离成微服务也会非常轻松。